

Data Analytics (PETR 6397)

Deep Computer Vision Using Convolutional neural Networks (Convnets)

Dr. Ahmad Sakhaee-Pour

Petroleum Engineering Department

University of Houston

Spring 2025

Discussed topics

- Filter
- Pooling
- Augmentation
- Convolutional layer
- Convolutional neural network
- Common designs
- Transfer learning

History of the convolutional neural network

- Convolutional neural networks (CNNs) emerged from the field of the visual cortex of the brain
- CNNs have benefited from signal processing in electrical engineering
- Researchers have used CNNs in image recognition for more than four decades
- CNNs have applications in search services, self-driving cars, and video classification

Visual cortex architecture

- Researchers showed that many neurons in the visual cortex have a small local receptive field (they are focused on a small local region)
- The receptive fields may overlap

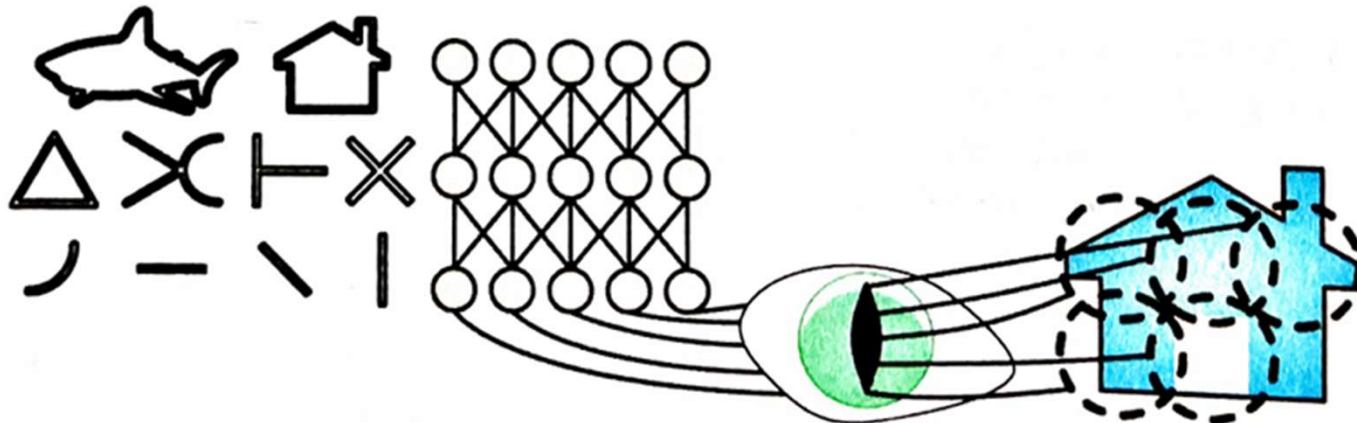
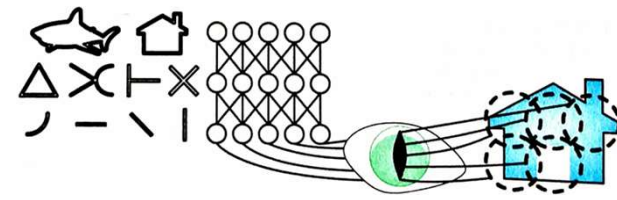


Fig. 14.1

- The receptive fields together cover the image

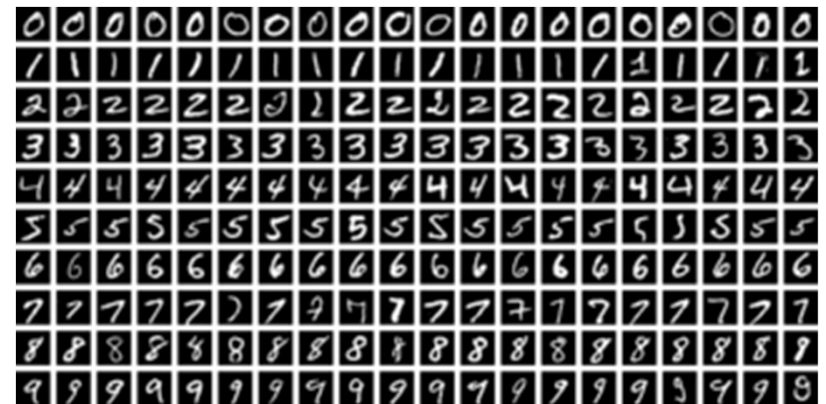


Behavior of the neurons

1. Some neurons react only to horizontal lines
 2. Some react only to vertical lines
 3. Some react to lines with different orientations
- 1-3 may have similar or different receptive fields
- Neurons with larger receptive fields react to more complex patterns based on lower-level patterns
 - Each neuron is connected to nearby neurons from the previous layer

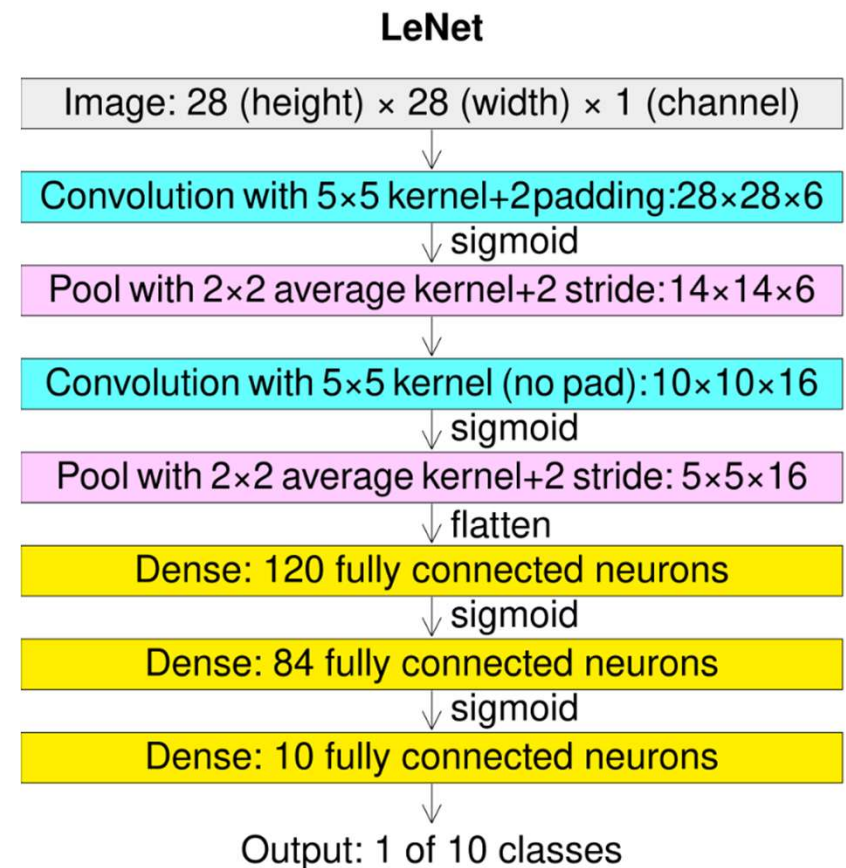
LeNet (1998)

- The visual cortex studies led to convolutional neural networks
- LeCun introduced the LeNet-5 to recognize handwritten digits
- LeNet-5 is used to recognize handwritten digits on checks
- We do not know a priori what controls the overall behavior



Architecture of LeNet-5

- It has **five** layers with tuning parameters (that is why we call it LeNet-**5**)
- It works fine for small images (28x28) but breaks down for larger images
- The computational power was not sufficient to use LeNet-5 for more complex problems



Convolutional layers

- Main idea: Neurons in the convolutional layer are not connected to all pixels (or neurons in the previous layer)
- Network first focuses on small, low-level features (layer 1)
- It then assembles the low-level features into high-level features (layer 2)

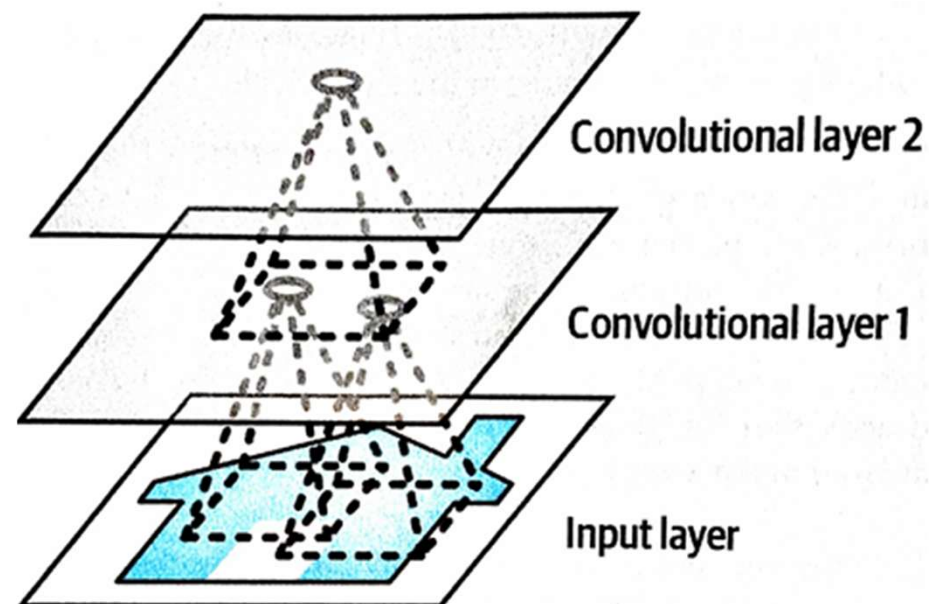
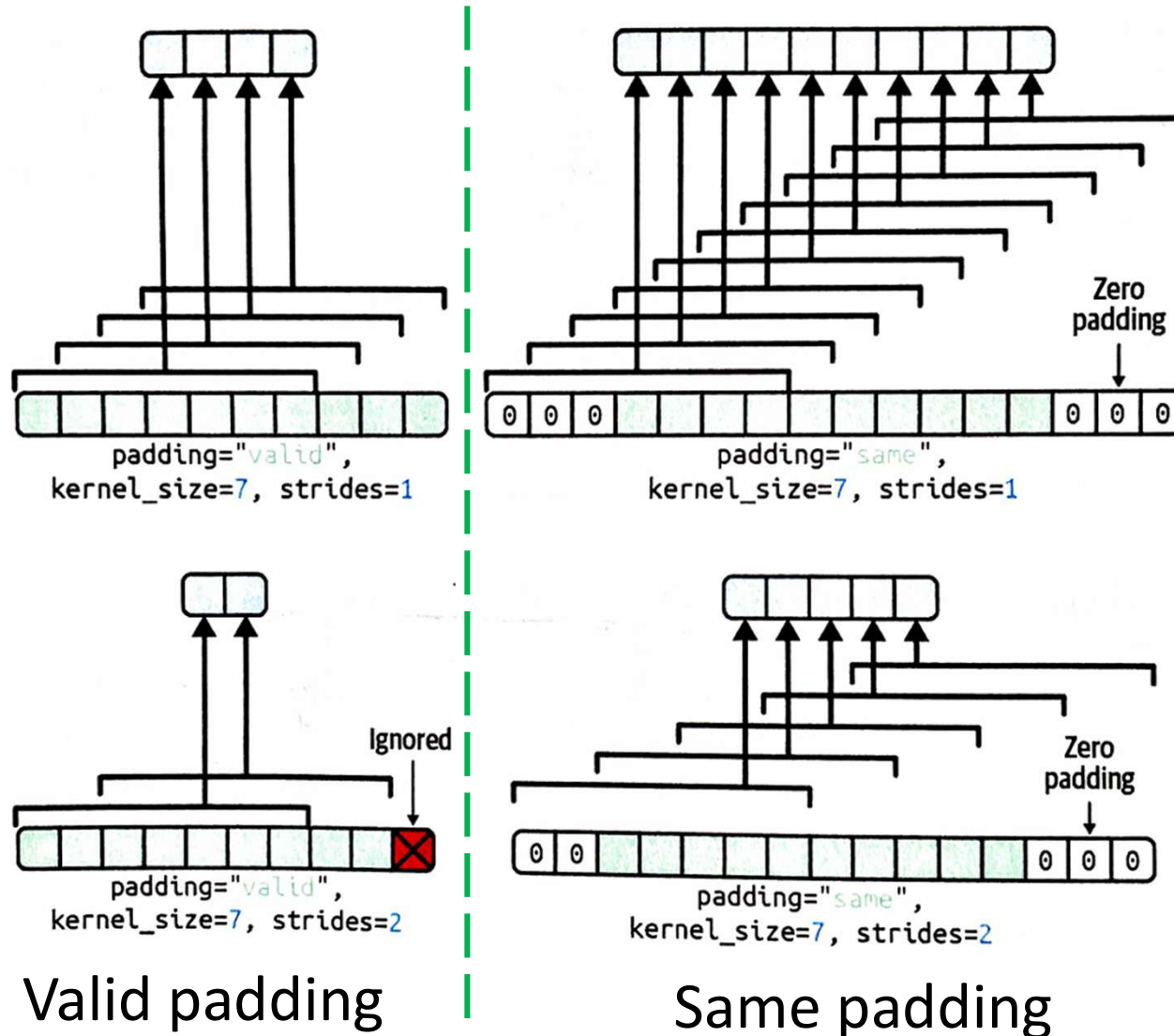


Fig. 14.2

Key properties of the convolutional layer

- Kernel size
- Stride
- Dilation
- Padding: Same padding (zero padding) vs valid padding

Valid padding vs. same padding



Effects of different parameters on the feature size

- Slide
- Kernel size
- Dilation
- Padding (same vs. valid)

Code

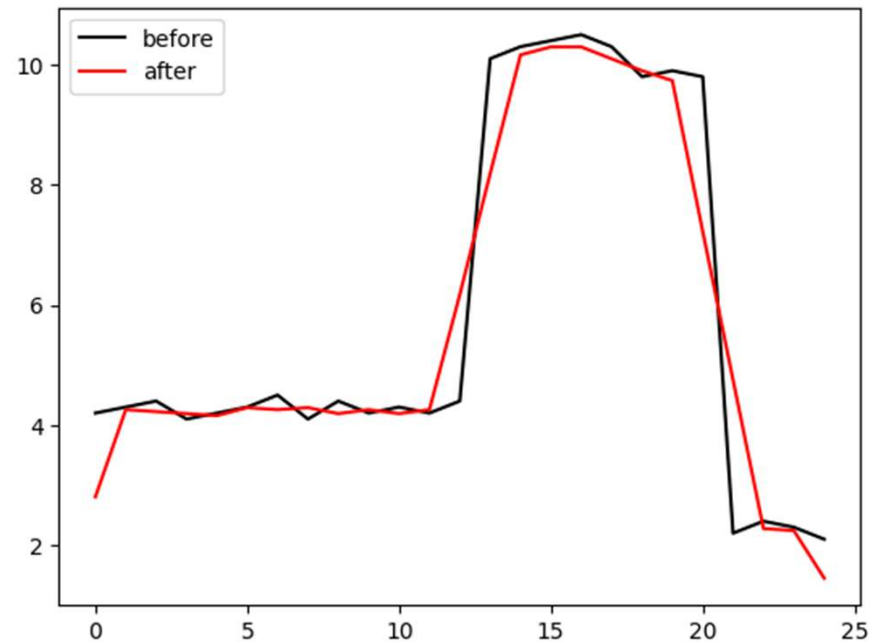
- Run 1d_convolution
- Calculate the result of convolution for 1D input with zero padding:

```
a=[4,5,6,7,8,9] #inputs
```

```
b=[1,2,3] #weights
```

Effects of first filter

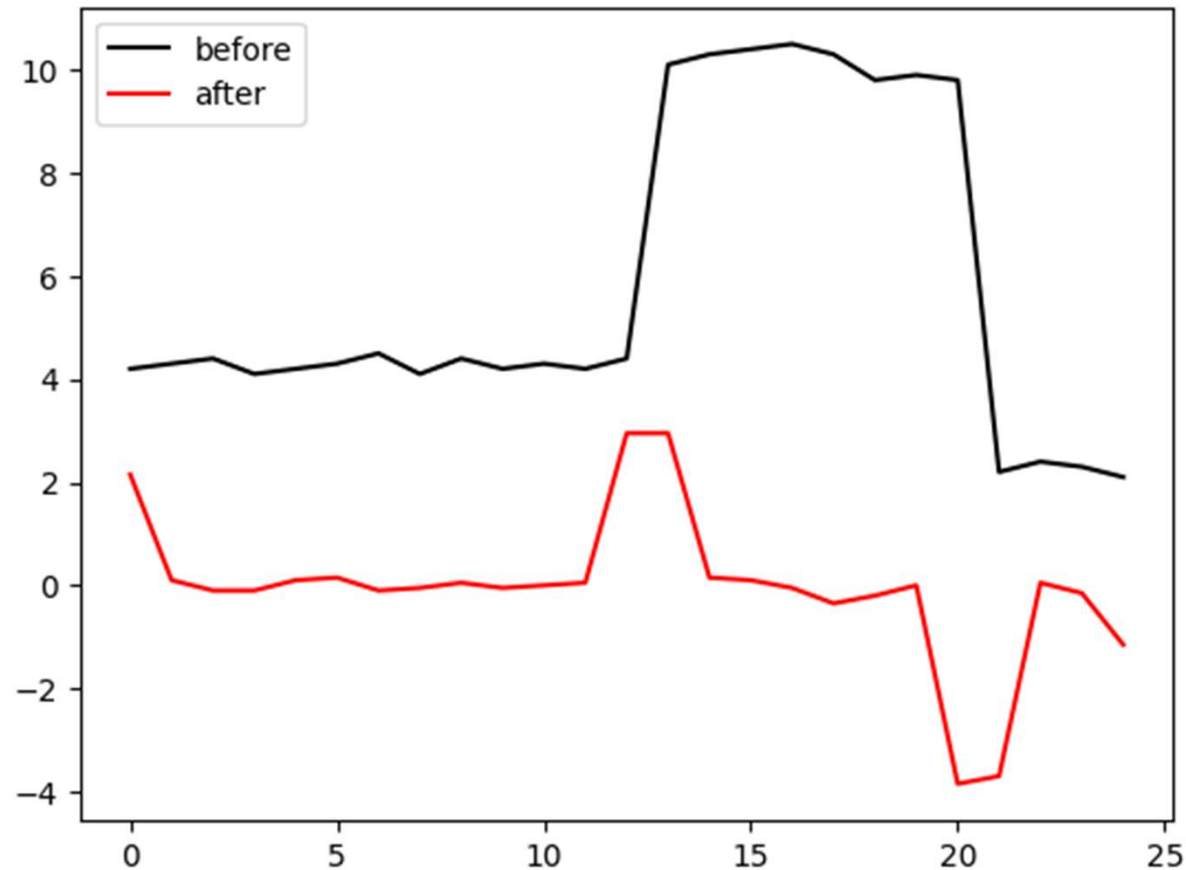
- Q1) What does the first filter do? Why?



- Why are the last and first values of the red curve lower?

Effects of second filter

- What does the second filter do? Why?



Effects of filtering more than once

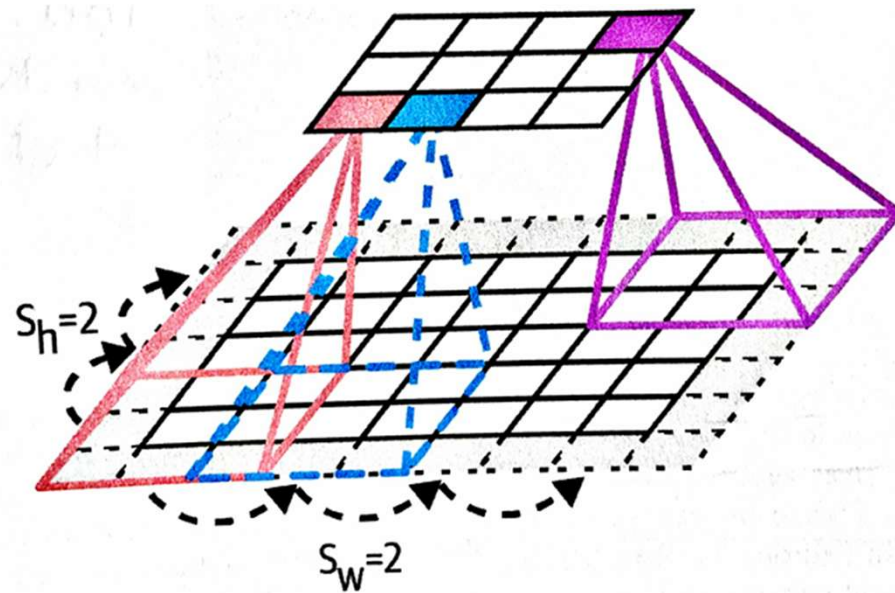
Q1) What do you expect to happen if we apply the first convolution twice? Can this be represented as a single convolution?

Q2) What do you expect to happen if we apply the last convolution twice? Can this be represented as a single convolution?

Q3) Do you see any activation function in the code? How would this change your answer to the previous questions?

Example of 2d convolution

- Determine the stride, dilation, kernel size, and padding type



Effects of filters on images

Filtering makes subsequent steps easier

- Sample the same points with a random noise

$$\text{Signal} = \text{Real value} + \text{noise}$$

- Sampling the same point with a random distribution:

$[10] + \text{noise}$

$[10] + \text{noise}$

$[10] + \text{noise}$

$[10] + \text{noise}$

$[10] + \text{noise}$

Noise with a random distribution and zero mean

$$5 \times 10 + (\text{noise} + \text{noise} + \text{noise} + \text{noise} + \text{noise}) \\ = (50 + \text{small value}) / 5$$

Common filtering techniques

- Mean distribution
- Gaussian distribution
- Median distribution

Original image of Lana

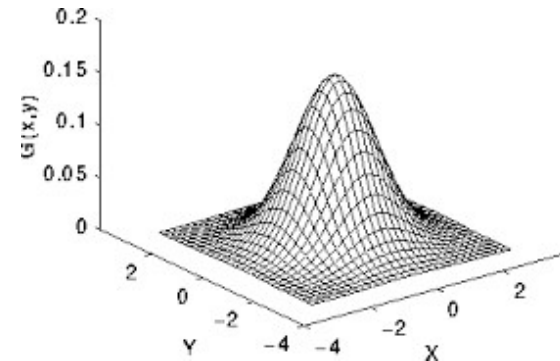


Applying Mean and Gaussian distributions

$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

$\frac{1}{273}$



After filtering



After filtering 21

Median filtering

- Examples: $\text{Median}[2\ 2\ 80] = 2$, $\text{Median}[2\ 6\ 80] = 6$,
 $\text{Median}[3\ 6\ 80] = 6$, $\text{Median}[3\ 3\ 6] = 3$



Original



After filtering

Applying vertical and horizontal filters using convolutional layers

Vertical and horizontal filters

- Filters (neuron weights) can be represented as a small image
- Zero values appear black
- Vertical lines are enhanced when we apply the vertical filter



Original



With vertical filter

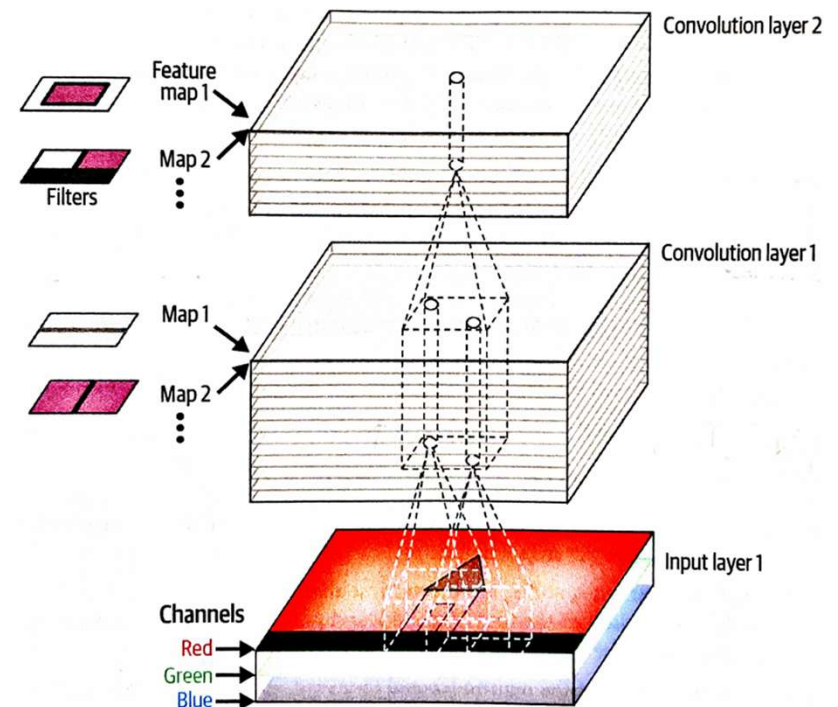


With horizontal filter

Q: How do we find the best filter (mean, median, ...)?

Stacking feature maps

- Each convolutional layer has multiple filters based on our design
- Each layer has one feature map for each filter
- We apply multiple filters to detect various features (horizontal lines, vertical lines, etc.)
- *discussion of calculations



Loading image and discussing its dimensions

Loading image:

```
images = load_sample_images()["images"]  
images = tf.keras.layers.CenterCrop(height=70,  
width=120)(images)  
images = tf.keras.layers.Rescaling(scale=1/255)(images)
```

images.shape:

```
TensorShape([2, 70, 120, 3])
```

Q: Why is it 4D? What does each dimension mean?

Implementing convolutional layers

```
tf.random.set_seed(42)
conv_layer = tf.keras.layers.Conv2D(filters=32,
kernel_size=7)
fmaps = conv_layer(images)
```

Q1: What is the meaning of filters=32? kernel_size=7?

```
fmaps.shape
TensorShape([2, 64, 114, 32])
```

```
images.shape:
TensorShape([2, 70, 120, 3])
```

Q2: How did the dimension change?

Q3: What would be the dimensions with padding = same?

Kernel dimensions

```
kernels.shape  
(7, 7, 3, 32)
```

```
(kernel_height, kernel_width, input_channels,  
 output_channels)
```

Q1: How are the weights initialized?

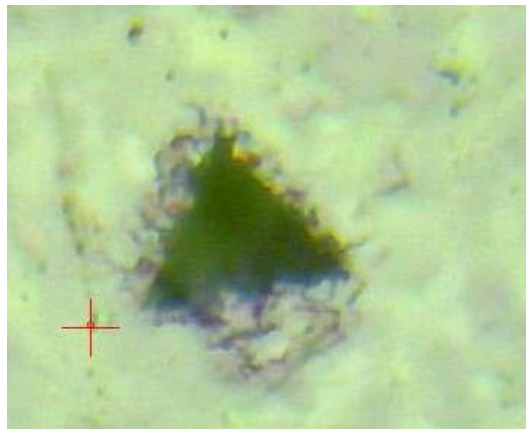
Q2: Is the stable initialization important here? Why?

Hands-on practice (second code)

- Open `resize_and_augment` code
- Read the image:

```
image= cv2.imread('/content/drive/MyDrive/3438.JPG')
```

- Visualize it: `cv2.imshow(...)`



(287, 358, 3)

Pooling layers

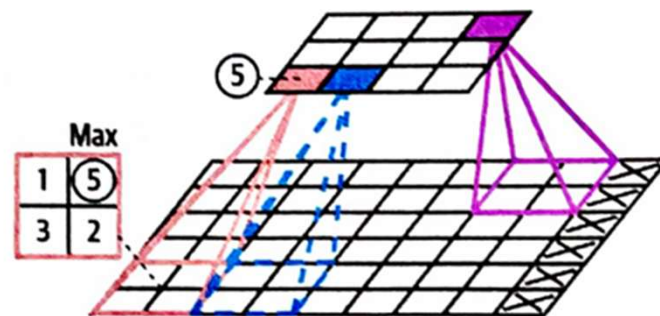
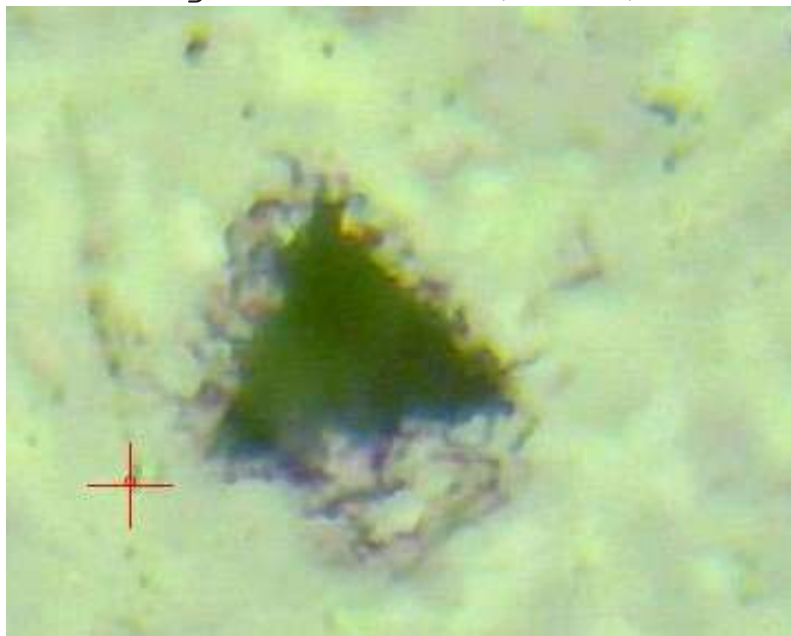
- It subsamples the inputs to reduce the computational cost
- Usually, it uses the strongest (dominant value) signal by pooling the maximum (maxpooling)
- We should specify its size, stride, and padding type

Apply pooling

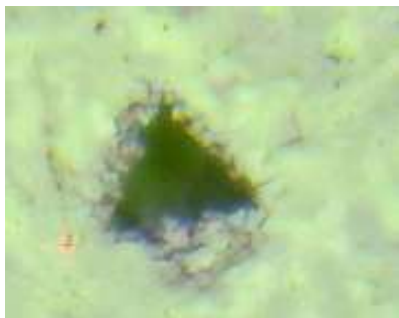
```
max_pool = tf.keras.layers.MaxPool2D(pool_size=2)
```

Pooling examples

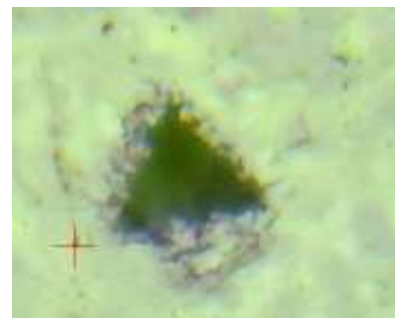
Original: (287, 358, 3)



Size=2, stride=2, padding=valid



Max Pooled: (143, 179, 3)



Average Pooled: (143, 179, 3)
32

Discussion of pooling

- Average pooling used to be popular, but maxpooling has become more common because it works better
- Q: What is the obvious negative impact of pooling on data?

Data Augmentation

Data Augmentation

- Main reason: Deep learning performs better as the number of images increases
- Researchers often augment the dataset
- Augmentation assumes that transformations do not alter the object represented in an image
- Augmentation also has applications in minority oversampling



original



Rotated 45 degrees

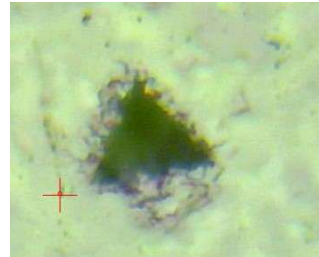
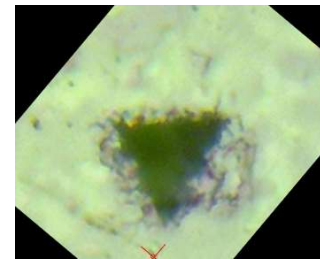
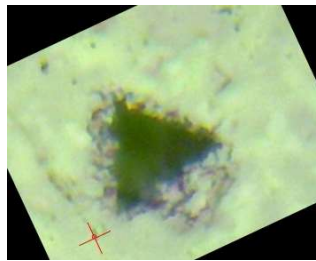
Common image augmentations

- Rotation
- Flipping
- Cropping
- Random translation
- Random brightness adjustment
- Random noise
-

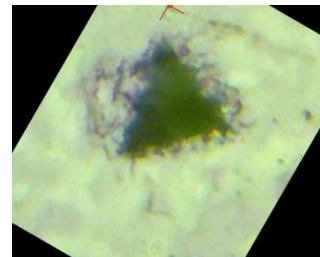
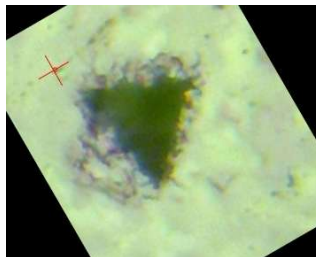
Implementing image augmentation

Rotating the image

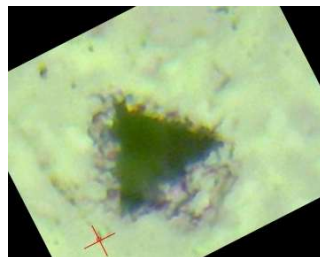
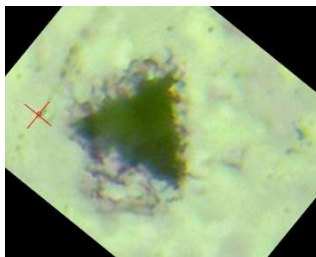
- Rotate 25, 50 degrees counterclockwise



- Rotate 60, 120, 240 degrees clockwise

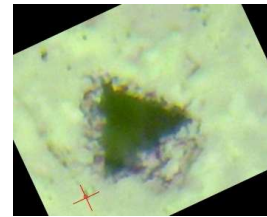


- Rotate randomly

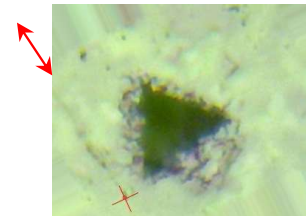


Filling the empty space after rotation

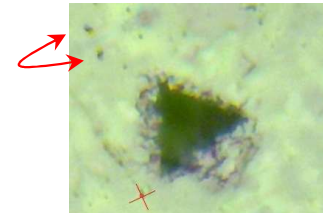
- 1. Constant color



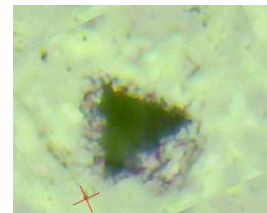
- 2. Replication



- 3. Reflection

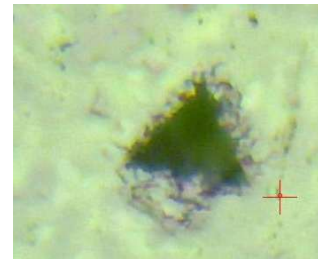


- 4. Wrapping

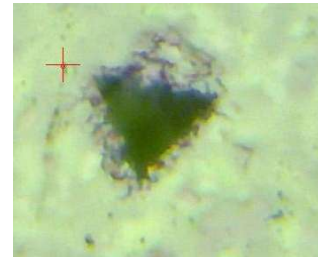


Flipping the image

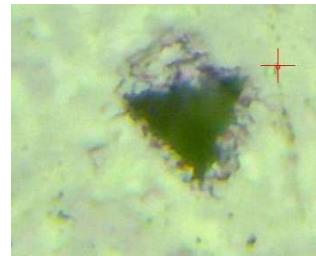
- Horizontal



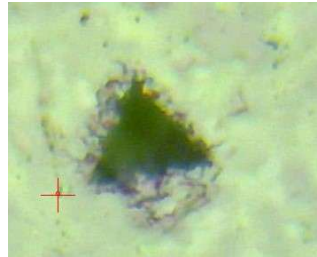
- Vertical



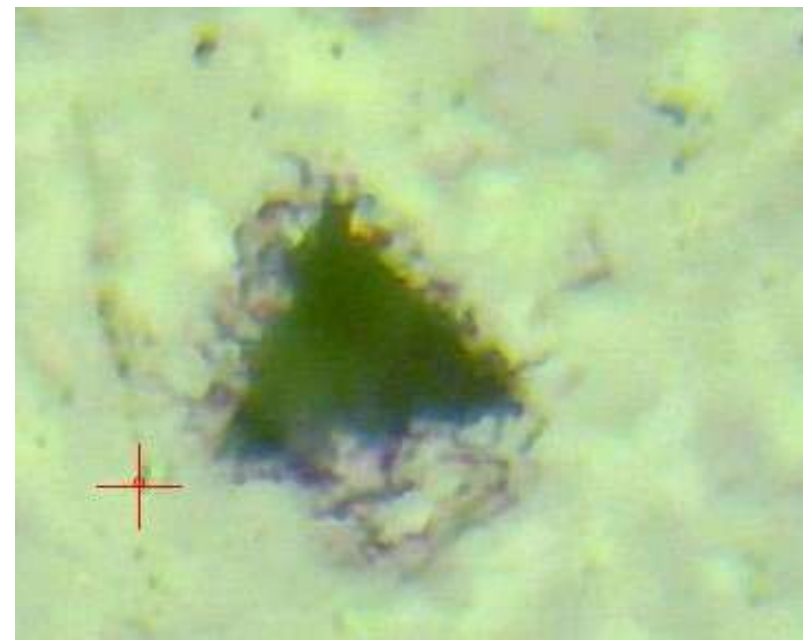
- Horizontal and vertical



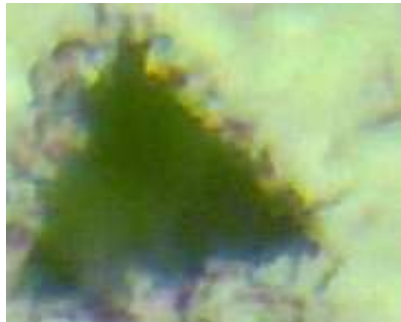
- Random



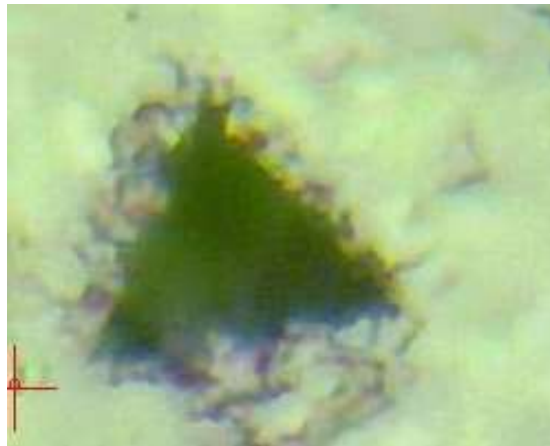
Cropping the image



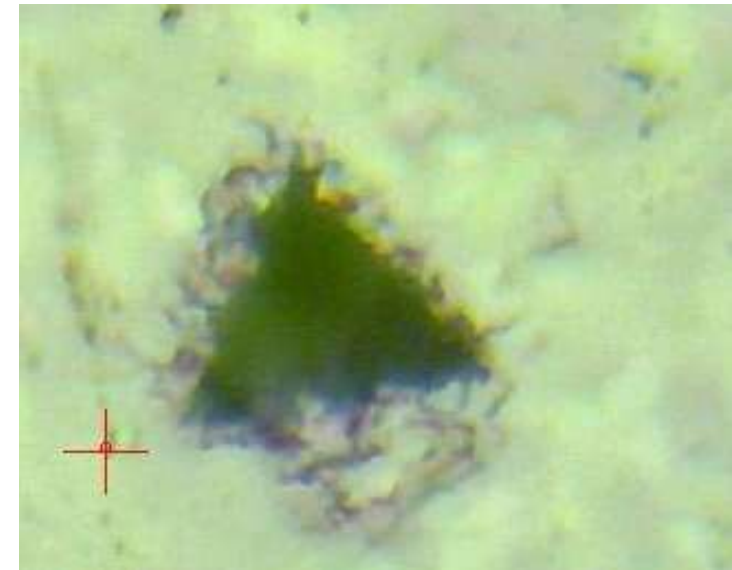
Original



Cropping
fraction:0.5

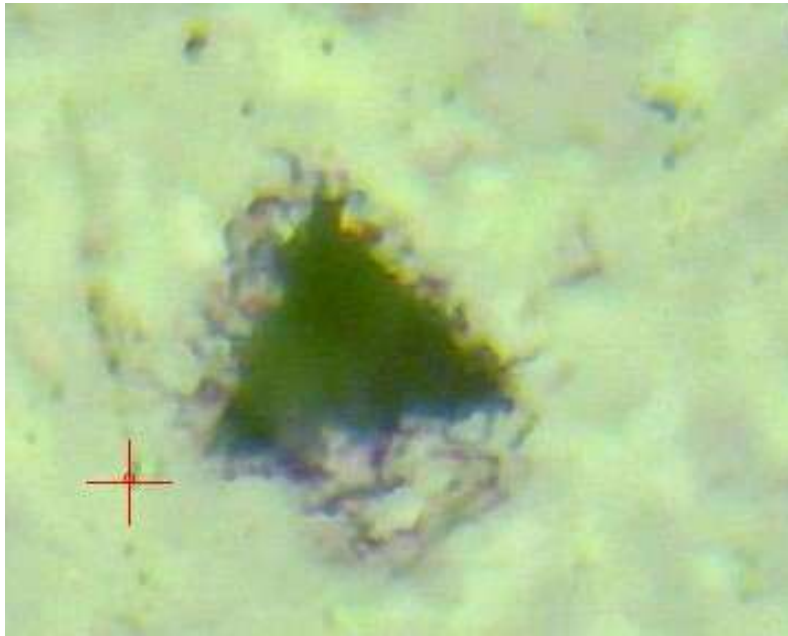


Cropping
fraction:0.7

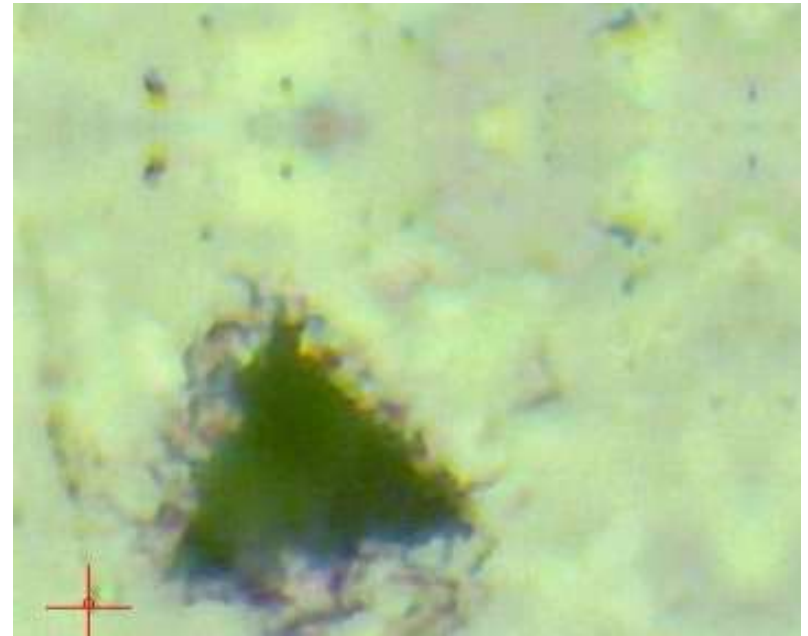


Cropping
fraction:0.9

Random translation

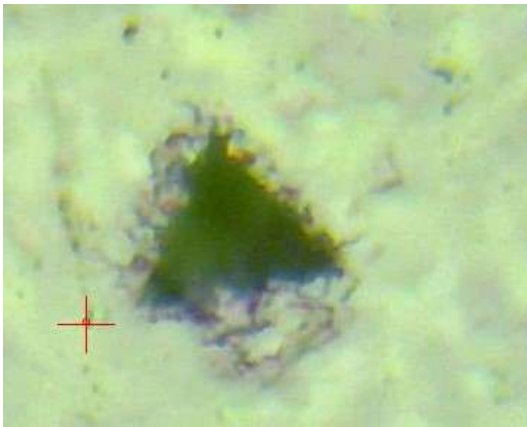


Original

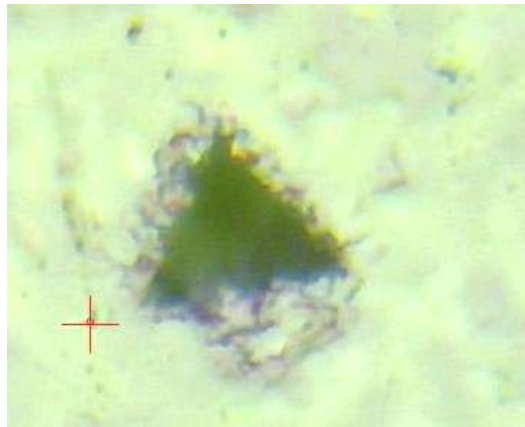


```
layers.RandomTranslation(0.2  
, 0.2, fill_mode="reflect",  
interpolation="bilinear")
```

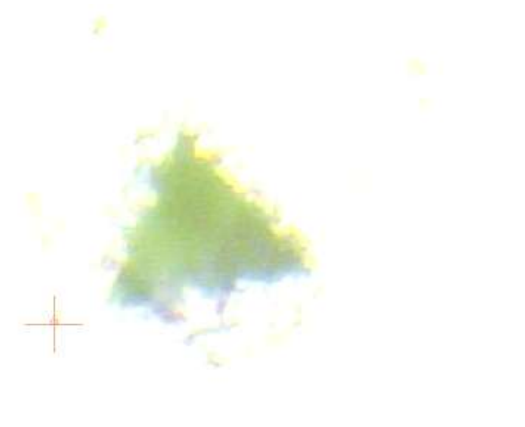
Random brightness adjustment



Original

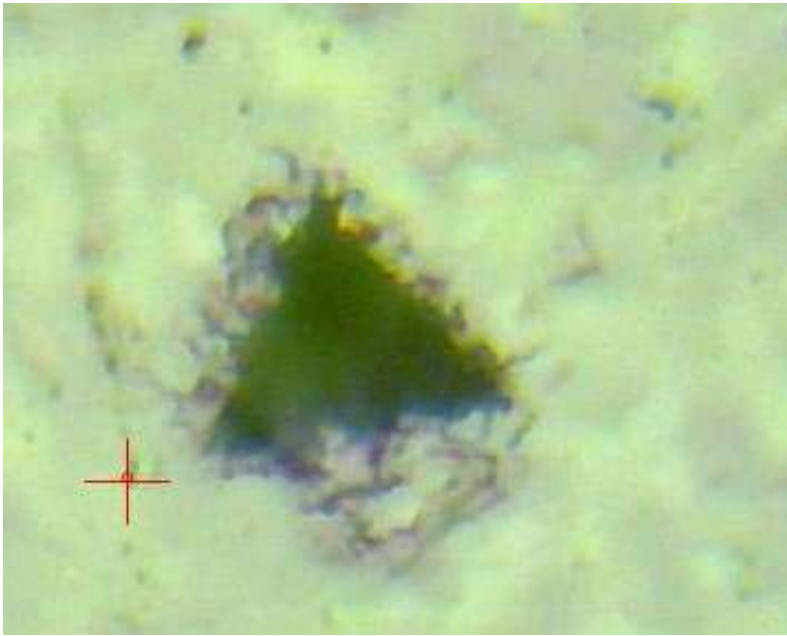


max_delta=0.25

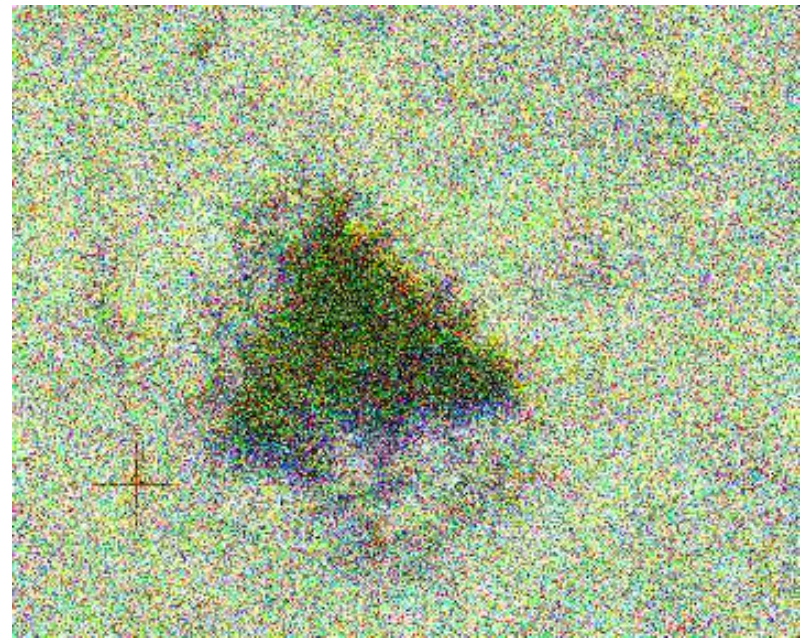


max_delta=0.50

Adding Gaussian noise



Original

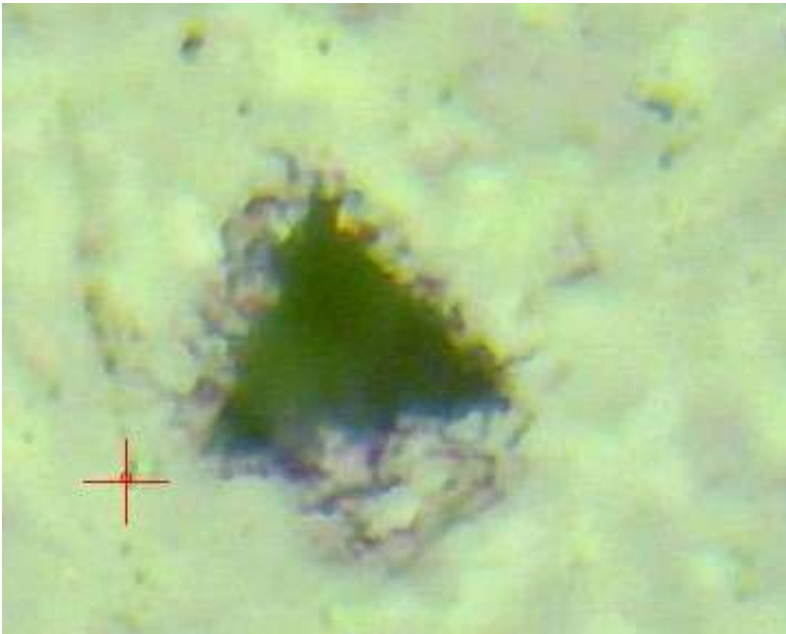


```
transform = A.Compose([  
    A.GaussNoise(var_limit=(10.0,  
50.0), p=0.5)])
```

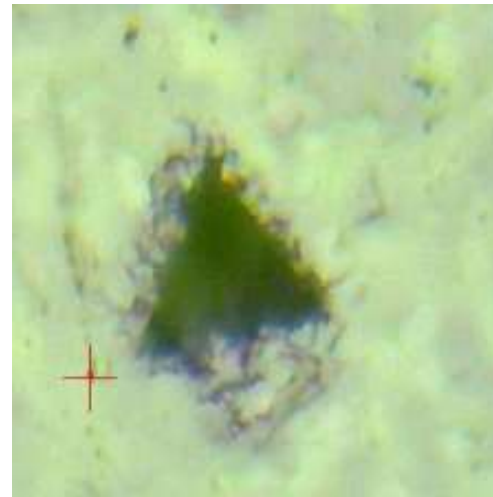
Image resizing is a common preprocessing

Resizing image

(287, 358, 3)



(224, 224, 3)

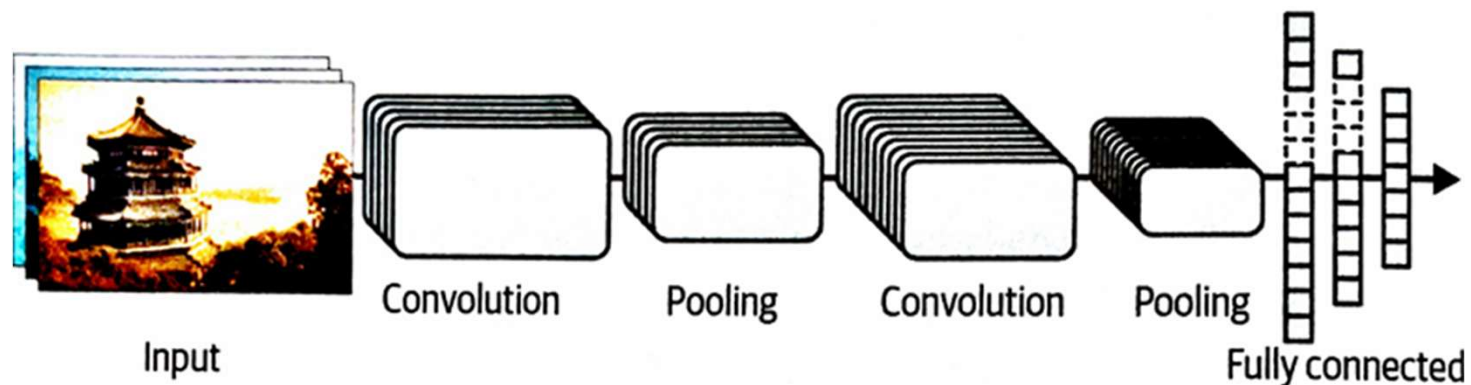


```
IMG_SIZE=224  
resize =  
tf.keras.Sequential([layers.Resizing(IMG_SIZE, IMG_SIZE)])
```

Convolutional neural networks

Convolutional neural network (CNN)

- Stack convolutional layers
- Alternate with pooling (usually maxpooling)
- 1 and 2 make the input size smaller
- We usually increase the number of filters downstream
- Include fully connected layers at the top



Implementation of CNN

```
DefaultConv2D = partial(tf.keras.layers.Conv2D, kernel_size=3,  
padding="same", activation="relu", kernel_initializer="he_normal")  
model = tf.keras.Sequential([  
    DefaultConv2D(filters=64, kernel_size=7, input_shape=[28, 28, 1]),  
    tf.keras.layers.MaxPool2D(),  
    DefaultConv2D(filters=128),  
    DefaultConv2D(filters=128),  
    tf.keras.layers.MaxPool2D(),  
    DefaultConv2D(filters=256),  
    DefaultConv2D(filters=256),  
    tf.keras.layers.MaxPool2D(),  
    tf.keras.layers.Flatten(),  
    tf.keras.layers.Dense(units=128, activation="relu",),  
    tf.keras.layers.Dropout(0.5),  
    tf.keras.layers.Dense(units=64, activation="relu",),  
    tf.keras.layers.Dropout(0.5),  
    tf.keras.layers.Dense(units=10, activation="softmax")  
])
```

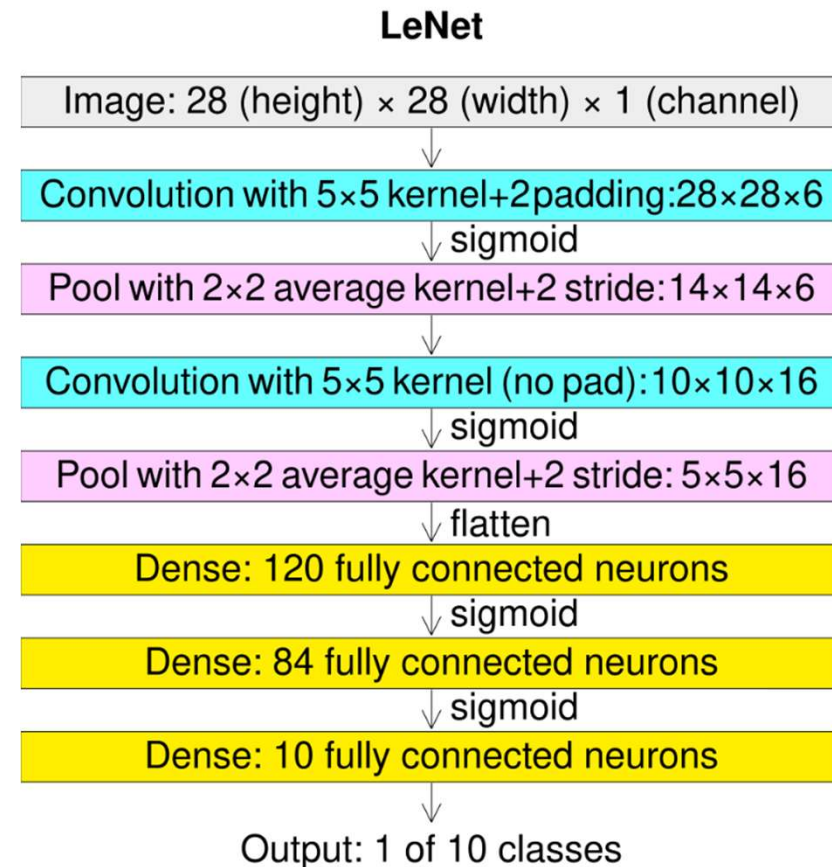
Discussion of the CNN architecture

- Q1: What should we do if the image size is not uniform (some are smaller, some are larger)?
- Q2: How to change the model architecture to analyze larger and more complex images?
- Q3: Why do we increase the number of filters downstream?

Common CNN architectures

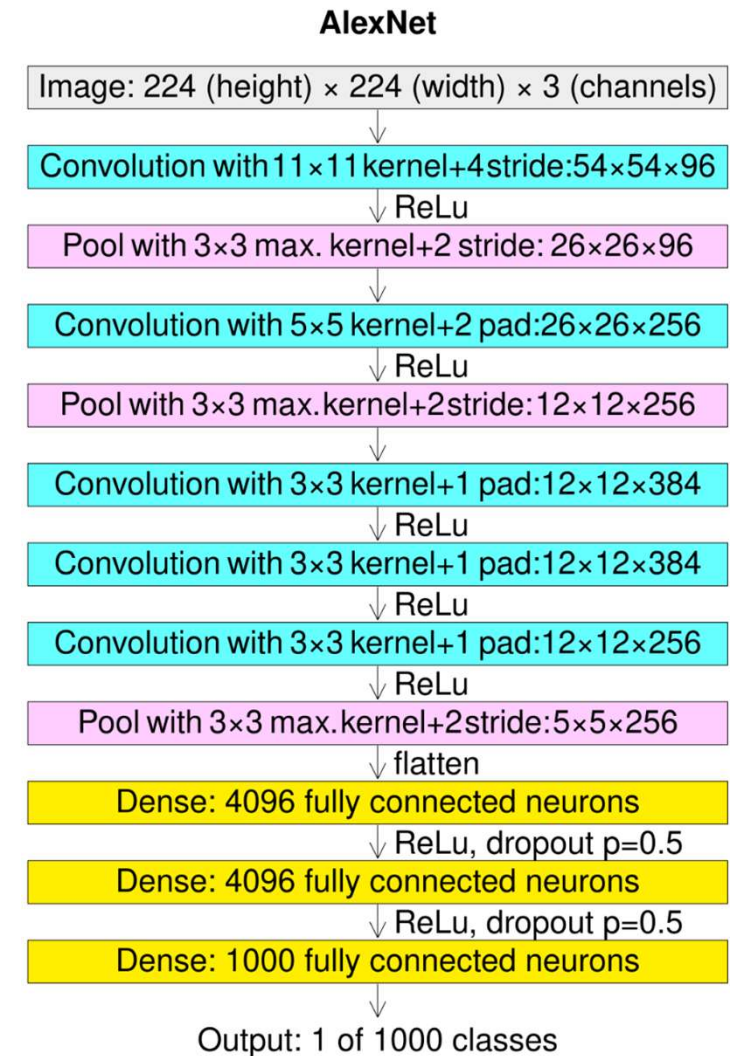
LeNet-5 has ~60,000 tuning parameters (1998)

Layer	Type	Maps	Size	Kernel size	Stride	Activation
Out	Fully connected	–	10	–	–	RBF
F6	Fully connected	–	84	–	–	tanh
C5	Convolution	120	1×1	5×5	1	tanh
S4	Avg pooling	16	5×5	2×2	2	tanh
C3	Convolution	16	10×10	5×5	1	tanh
S2	Avg pooling	6	14×14	2×2	2	tanh
C1	Convolution	6	28×28	5×5	1	tanh
In	Input	1	32×32	–	–	–



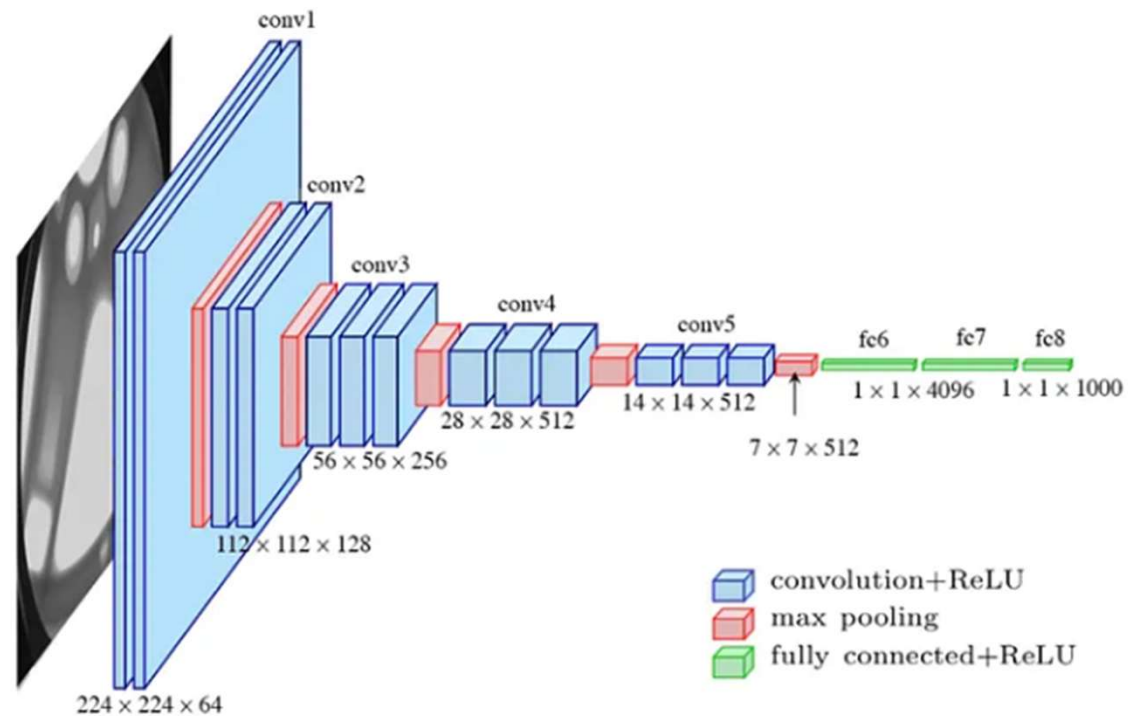
AlexNet has ~60 million tuning parameters (2012)

Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
Out	Fully connected	—	1,000	—	—	—	Softmax
F10	Fully connected	—	4,096	—	—	—	ReLU
F9	Fully connected	—	4,096	—	—	—	ReLU
S8	Max pooling	256	6×6	3×3	2	valid	—
C7	Convolution	256	13×13	3×3	1	same	ReLU
C6	Convolution	384	13×13	3×3	1	same	ReLU
C5	Convolution	384	13×13	3×3	1	same	ReLU
S4	Max pooling	256	13×13	3×3	2	valid	—
C3	Convolution	256	27×27	5×5	1	same	ReLU
S2	Max pooling	96	27×27	3×3	2	valid	—
C1	Convolution	96	55×55	11×11	4	valid	ReLU
In	Input	3 (RGB)	227×227	—	—	—	—



VGG16 has ~ 138 million tuning parameters (2014)

	Layer	Feature Map	Size	Kernel Size	Stride	Activation
Input	Image	1	224 x 224 x 3	-	-	-
1	2 X Convolution	64	224 x 224 x 64	3x3	1	relu
	Max Pooling	64	112 x 112 x 64	3x3	2	relu
3	2 X Convolution	128	112 x 112 x 128	3x3	1	relu
	Max Pooling	128	56 x 56 x 128	3x3	2	relu
5	2 X Convolution	256	56 x 56 x 256	3x3	1	relu
	Max Pooling	256	28 x 28 x 256	3x3	2	relu
7	3 X Convolution	512	28 x 28 x 512	3x3	1	relu
	Max Pooling	512	14 x 14 x 512	3x3	2	relu
10	3 X Convolution	512	14 x 14 x 512	3x3	1	relu
	Max Pooling	512	7 x 7 x 512	3x3	2	relu
13	FC	-	25088	-	-	relu
14	FC	-	4096	-	-	relu
15	FC	-	4096	-	-	relu
Output	FC	-	1000	-	-	Softmax

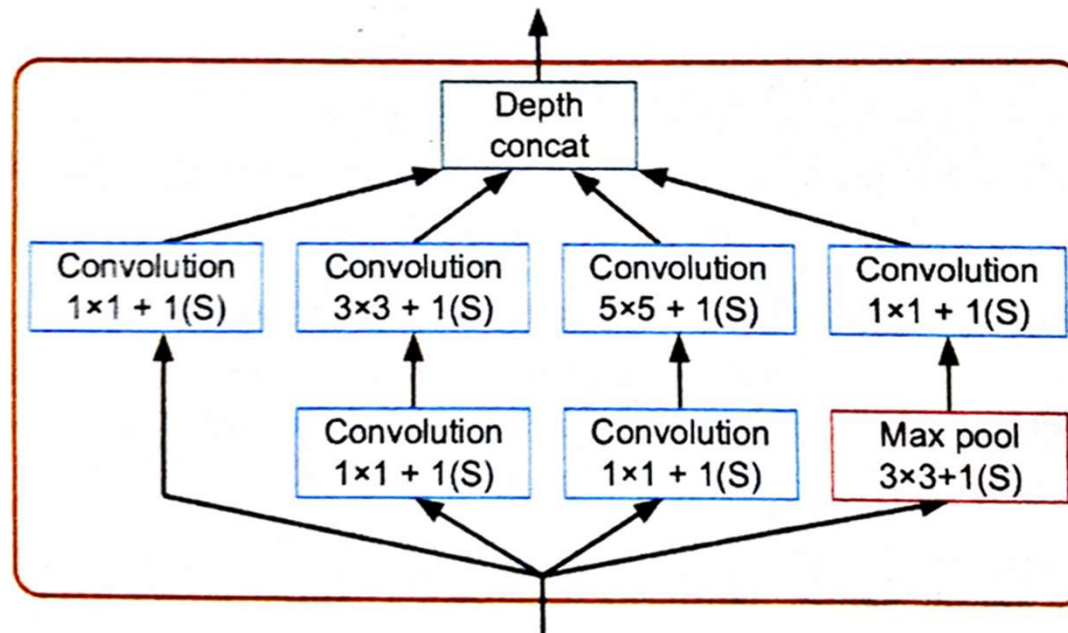


Evolution of deep learning

- Researchers made models deeper and bigger:
 - LeNet-5 (1998): 60,000 tuning parameters
 - AlexNet (2012): 60 million tuning parameters
 - VGG16 (2014): 138 million tuning parameters
- Researchers also developed new architectures:
 - Inception (2014)
 - Xception (2016)
 - ...
- More recently, researchers have focused on generative models

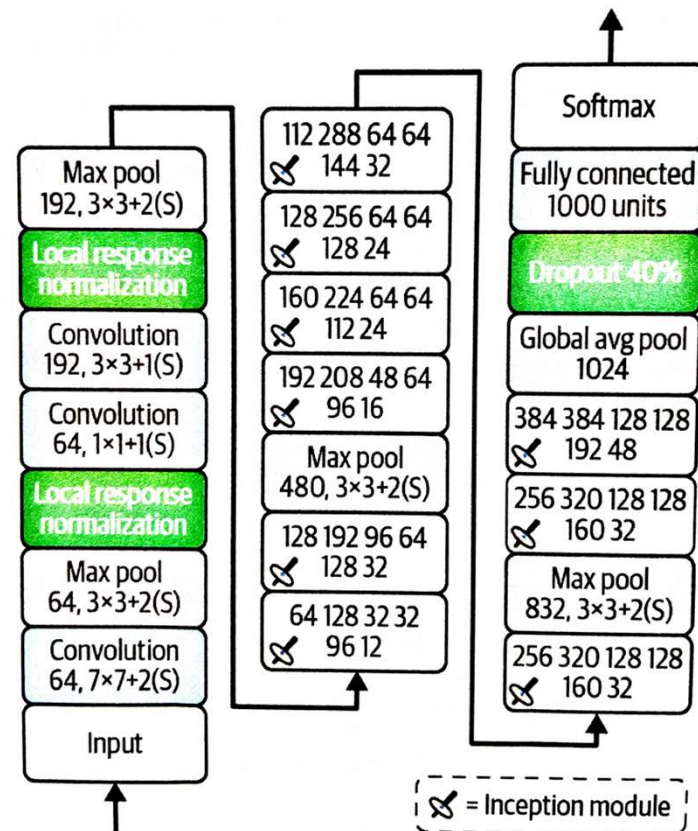
Inception was a smarter design used in GoogLeNet

- GoogLeNet uses subnetworks called “inception modules”
- GoogLeNet outperforms AlexNet
- GoogLeNet uses its tuning parameters more wisely



Inception module

GoogLeNet has ~6 million tuning parameters
(one-tenth of AlexNet)



Discussion

- These models are empirical
- How would you use an empirical approach for your problem?
- Would you design your own unique model?

Available architectures

- Listed models are empirical and sometimes very different
- They are easily accessible (keras.io/api/applications/)
- It would be wise to test successful models first before designing a new one

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8
DenseNet121	33	75.0%	92.3%	8.1M	242	77.1	5.4
DenseNet169	57	76.2%	93.2%	14.3M	338	96.4	6.3
DenseNet201	80	77.3%	93.6%	20.2M	402	127.2	6.7
NASNetMobile	23	74.4%	91.9%	5.3M	389	27.0	6.7

Finding the best architecture

- The listed models can be called easily (shown later)
- We do not need to develop them from scratch, but we should be able to use them for high-quality data.
- Larger models usually perform better, but this is not always true (GoogLeNet vs AlexNet)
- Which model would you choose for your problem?

Common preprocessing for using pretrained models

1. **Resize** the input

2. **Shuffle**

3. **Augment** the inputs to improve the performance

```
preprocess = tf.keras.Sequential([  
    tf.keras.layers.Resizing(height=224, width=224,  
        crop_to_aspect_ratio=True),  
    ])
```

```
train_set = train_set.shuffle(1000,  
    seed=42).batch(batch_size).prefetch(1)
```

```
data_augmentation = tf.keras.Sequential([  
    tf.keras.layers.RandomFlip(mode="horizontal", seed=42),  
    tf.keras.layers.RandomRotation(factor=0.05, seed=42),  
    tf.keras.layers.RandomContrast(factor=0.2, seed=42)  
    ])
```

Loading the pretrained model Xception

```
base_model =  
tf.keras.applications.xception.Xception(weights="imagenet",  
                                         include_top=False)  
  
avg =  
tf.keras.layers.GlobalAveragePooling2D()(base_model.output)  
output =  
tf.keras.layers.Dense(n_classes, activation="softmax")(avg)  
model = tf.keras.Model(inputs=base_model.input,  
                        outputs=output)
```

- We usually replace the top part because our problem is different from the original one
- We may add more fully connected layers to improve the performance of the model

Two examples of using the pretrained model (Transfer learning)

First: keep all the pretrained weights fixed

```
for layer in base_model.layers:  
    layer.trainable = False
```

Second: make the weights of layers 50 and above trainable
(this is called unfreezing)

```
for layer in base_model.layers [50:]:  
    layer.trainable = True
```